

BRIXCOT Lead Generation Platform - Business Workflows & Integrations

Document 2 of 3: Core Business Logic, Payment Flows, and Integration Capabilities

1. CORE BUSINESS WORKFLOWS

A. Lead Generation & Capture Flow

1. Form Creation (Seller)

Component: `app/dashboard/seller/lead_management/formbuilder/page.tsx`

Seller Dashboard → Lead Management → Form Builder

└> Define Form Details

- Form Name
- Industry (dropdown)
- Lead Source (optional)

└> Add Form Fields (Drag & Drop)

- Field Types:
 - Text Input
 - Email (with validation)
 - Phone (with formatting)
 - Select Dropdown
 - Checkbox
 - Radio Buttons
 - Textarea
 - Number
 - Date
- Configure per field:
 - Label
 - Required (yes/no)
 - Placeholder
 - Options (for select/radio/checkbox)
 - Validation rules

└> Style Configuration (Optional)

- Primary Color
- Button Text
- Success Message

└> Save Form

```
POST /api/form
{
  formName, industry, leadSource,
  fields: [{ id, type, label, required, options }]
}
```

└> Generate Embed Code

- Unique formId (UUID)
- Embed URL: /forms/{formId}
- iFrame code
- Direct link

Database Operations:

```
// Create Form document
const form = await Form.create({
  formId: uuidv4(),
  userId: session.user.id,
  formName,
  industry,
  fields,
  createdAt: new Date(),
});

// Update User's forms array
await User.findByIdAndUpdate(session.user.id, {
  $push: { forms: form._id },
});
```

2. Lead Submission Flow

Component: app/forms/[formId]/page.tsx

API: app/api/form/submit/route.ts

User visits form URL (/forms/{formId})

|

└─> Fetch Form Configuration

|

GET /api/form/{formId}

|

- Retrieve fields, styling, validation rules

|

└─> User Fills Form

|

- Client-side validation

|

- Required field checks

|

- Email/phone format validation

|

└─> Submit Form

|

POST /api/form/submit

|

{

|

formId,

|

fields: { name, email, phone, company, ... }

|

}

|

└─> Server Processing

|

└─> Validate Data

|

- Check required fields

|

- Validate email format

|

- Sanitize inputs

|

└─> Create Lead Document

|

const lead = await Lead.create({

|

userId: form.userId,

|

formId,

|

name, email, phone, company,

|

fields: formData,

|

status: "new",

|

industry: form.industry,

|

leadSource: form.leadSource,

|

createdAt: new Date()

|

});

|

└─> Calculate Lead Score

|

calculateLeadScore(lead) →

|

scoreFactors = {

|

completeness: 0-3 points,

|

responsiveness: 0-3 points,

|

valuePotential: 0-4 points

|

}

```

| |   leadScore = sum(scoreFactors) / 10 * 10
| |   qualificationScore = leadScore * 10
| |
| |   ↳ Update Form's submittedLeads
| |   await Form.findByIdAndUpdate(formId, {
| |     $push: { submittedLeads: lead._id }
| |   });
| |
| |   ↳ Trigger Automation Workflows
| |   - Find active workflows with "lead_received" trigger
| |   - Check trigger conditions
| |   - Execute actions (email, SMS, assignment)
| |
| |   ↳ Send Notifications
| |   - Notify seller of new lead
| |   - If auto-distribution enabled → assign immediately
|
| ↳ Return Success
|   - Display thank you message
|   - Track form conversion

```

Lead Scoring Algorithm:

```

function calculateLeadScore(lead) {
  let completeness = 0;
  let responsiveness = 0;
  let valuePotential = 0;

  // Completeness (0-3 points)
  const requiredFields = ["name", "email", "phone"];
  const optionalFields = ["company", "address", "industry"];

  const filledRequired = requiredFields.filter((f) => lead[f]).length;
  completeness = (filledRequired / requiredFields.length) * 3;

  const filledOptional = optionalFields.filter((f) => lead[f]).length;
  completeness += (filledOptional / optionalFields.length) * 1;

  // Responsiveness (0-3 points)
  const submissionTime = new Date(lead.createdAt);
  const businessHours = isBusinessHours(submissionTime); // 9am-5pm weekdays
  if (businessHours) responsiveness += 2;
  if (lead.fields?.find((f) => f.label === "Best Time to Contact"))
    responsiveness += 1;

  // Value Potential (0-4 points)
  if (lead.company) valuePotential += 1;
  if (lead.industry === "enterprise") valuePotential += 2;
  if (lead.fields?.find((f) => f.label === "Budget" && f.value > 10000))
    valuePotential += 1;

  const totalScore = completeness + responsiveness + valuePotential;
  const leadScore = Math.round((totalScore / 10) * 10); // 0-10 scale
  const qualificationScore = leadScore * 10; // 0-100 scale

  return {
    leadScore,
    qualificationScore,
    scoreFactors: { completeness, responsiveness, valuePotential },
  };
}

```

3. Lead Qualification

Automatic qualification based on score:

```
Lead Created
|
|→ Calculate qualificationScore (0-100)
|
|→ Check Against Threshold
|   if (qualificationScore >= seller.qualificationThreshold || 50) {
|     lead.status = "qualified"
|   } else {
|     lead.status = "unqualified"
|   }
|
|→ If Qualified
|   |→ Mark as "available" for marketplace (if not exclusive)
|   |→ Trigger distribution (if auto-distribution enabled)
|   |→ Notify potential buyers matching criteria
|
|→ If Unqualified
|   - Notify seller for manual review
|   - Seller can manually qualify or discard
```

B. Lead Distribution Workflows

1. Manual Distribution

Component: app/dashboard/seller/lead_management/page.tsx

API: app/api/leads/assignExclusiveLead/route.ts

Seller views leads list

```
|
|
|→ Filter & Sort Leads
|  - Status: "new", "qualified"
|  - Industry
|  - Date range
|  - Lead score
|
|
|→ Select Lead(s)
|  - Single select
|  - Bulk select (checkbox)
|
|
|→ Click "Assign to Buyer"
|  - Modal opens with buyer list
|  - Filter buyers by:
|    • Status: "active"
|    • Service locations match lead.location
|    • Industries include lead.industry
|
|
|→ Select Buyer(s)
|  - Single buyer (exclusive)
|  - Multiple buyers (shared)
|
|
|→ Configure Assignment
|  - Set as exclusive (yes/no)
|  - Add notes
|  - Set expiration time
|
|
|→ Submit Assignment
|  POST /api/leads/assignExclusiveLead
|  {
|    leadId,
|    buyerIds: [buyerId1, buyerId2],
|    exclusive: true,
|    notes: "High-value lead"
|  }
|
|
|→ Update Lead Document
|  await Lead.findByIdAndUpdate(leadId, {
|    status: "assigned",
|    exclusive: true,
|    $push: {
|      assignedTo: buyerIds.map(id => ({
```



```

|         id: uuidv4(),
|         buyerId: id,
|         accepted: false,
|         rejected: false,
|         assignedAt: new Date(),
|         notes
|     )))
| }
| });
|
|→ Create Notifications
|   for each buyer:
|       await Notification.create({
|           userId: buyerId,
|           type: "lead_assigned",
|           message: "New lead assigned: {lead.name}",
|           leadId,
|           createdAt: new Date()
|       });
|
|→ Send Communications
|   - Email notification to buyer(s)
|   - SMS notification (if enabled)
|   - Dashboard notification badge
|
|→ Log Activity
|   - Audit trail
|   - Seller dashboard activity feed

```

2. Round Robin Distribution

Automatic equal distribution

API: `app/api/leads/distribute/roundrobin/route.ts` (internal)

Qualified Lead Created

```
|
|
|→ Check seller.leadDistributionMethod === "roundRobin"
|
|
|→ Get Active Buyers
|  const buyers = await Buyer.find({
|    registeredWith: sellerId,
|    status: "active",
|    industries: { $in: [lead.industry] },
|    $or: [
|      { 'serviceLocations.state': lead.location.state },
|      { 'serviceLocations.city': lead.location.city }
|    ],
|    currentLeadsToday: { $lt: '$maxLeadsPerDay' }
|  });
|
|
|→ Sort by Last Assignment
|  buyers.sort((a, b) => {
|    const aLastAssigned = a.lastAssignedAt || new Date(0);
|    const bLastAssigned = b.lastAssignedAt || new Date(0);
|    return aLastAssigned - bLastAssigned; // Oldest first
|  });
|
|
|→ Assign to Next Buyer
|  const selectedBuyer = buyers[0];
|
|  await Lead.findByIdAndUpdate(leadId, {
|    status: "assigned",
|    $push: {
|      assignedTo: {
|        buyerId: selectedBuyer._id,
|        assignedAt: new Date()
|      }
|    }
|  });
|
|
|→ Update Buyer Stats
|  await Buyer.findByIdAndUpdate(selectedBuyer._id, {
|    currentLeadsToday: selectedBuyer.currentLeadsToday + 1,
|    lastAssignedAt: new Date()
|  });
|
|
|→ Notify Buyer
```

```
| - Send notification
| - Email/SMS alert
|
└> Increment Round Robin Index
    await User.findByIdAndUpdate(sellerId, {
      $inc: { 'roundRobinIndex': 1 }
    });
```

3. Weighted Distribution

Priority-based distribution

Qualified Lead Created

```
|
|
|→ Get Active Buyers with Priorities
|   const buyers = await Buyer.find({
|     ...matchingCriteria,
|     priority: { $exists: true, $gte: 1 }
|   });
|
|
|→ Calculate Weights
|   const totalPriority = buyers.reduce((sum, b) => sum + b.priority, 0);
|
|   const weights = buyers.map(buyer => ({
|     buyer,
|     weight: buyer.priority / totalPriority,
|     cumulativeWeight: 0
|   }));
|
|   // Calculate cumulative weights
|   let cumulative = 0;
|   weights.forEach(w => {
|     cumulative += w.weight;
|     w.cumulativeWeight = cumulative;
|   });
|
|
|→ Random Selection (Weighted)
|   const random = Math.random(); // 0 to 1
|
|   const selectedBuyer = weights.find(w =>
|     random <= w.cumulativeWeight
|   ).buyer;
|
|   // Example:
|   // Buyer A: priority 8 → 8/15 = 0.53 (53% chance)
|   // Buyer B: priority 5 → 5/15 = 0.33 (33% chance)
|   // Buyer C: priority 2 → 2/15 = 0.13 (13% chance)
|
|
|→ Assign Lead
|   await assignLeadToBuyer(leadId, selectedBuyer._id);
|
|
|→ Track Statistics
|   - Log distribution decision
|   - Update buyer assignment count
```

4. Priority Distribution

Highest priority buyer gets lead first

Qualified Lead Created

```
|
|
|→ Get Buyers Sorted by Priority
|   const buyers = await Buyer.find({
|     ...matchingCriteria
|   }).sort({ priority: -1 }); // Highest first
|
|
|→ Assign to Highest Priority Buyer
|   const primaryBuyer = buyers[0];
|
|   await Lead.findByIdAndUpdate(leadId, {
|     status: "assigned",
|     $push: {
|       assignedTo: {
|         buyerId: primaryBuyer._id,
|         assignedAt: new Date(),
|         expiresAt: new Date(Date.now() + 24*60*60*1000) // 24 hours
|       }
|     }
|   });
|
|
|→ Set Expiration Timer
|   - If buyer doesn't accept within 24 hours
|   - Auto-reassign to next priority buyer
|
|   setTimeout(async () => {
|     const assignment = lead.assignedTo.find(a => a.buyerId === primaryBuyer._id);
|
|     if (!assignment.accepted && !assignment.rejected) {
|       // Mark as expired and reassign
|       await reassignLeadToNextBuyer(leadId, buyers[1]._id);
|     }
|   }, 24 * 60 * 60 * 1000);
|
|
|→ Notify Buyer with Urgency
|   - "You have 24 hours to accept this lead"
|   - Push notification
|   - Email with countdown
```

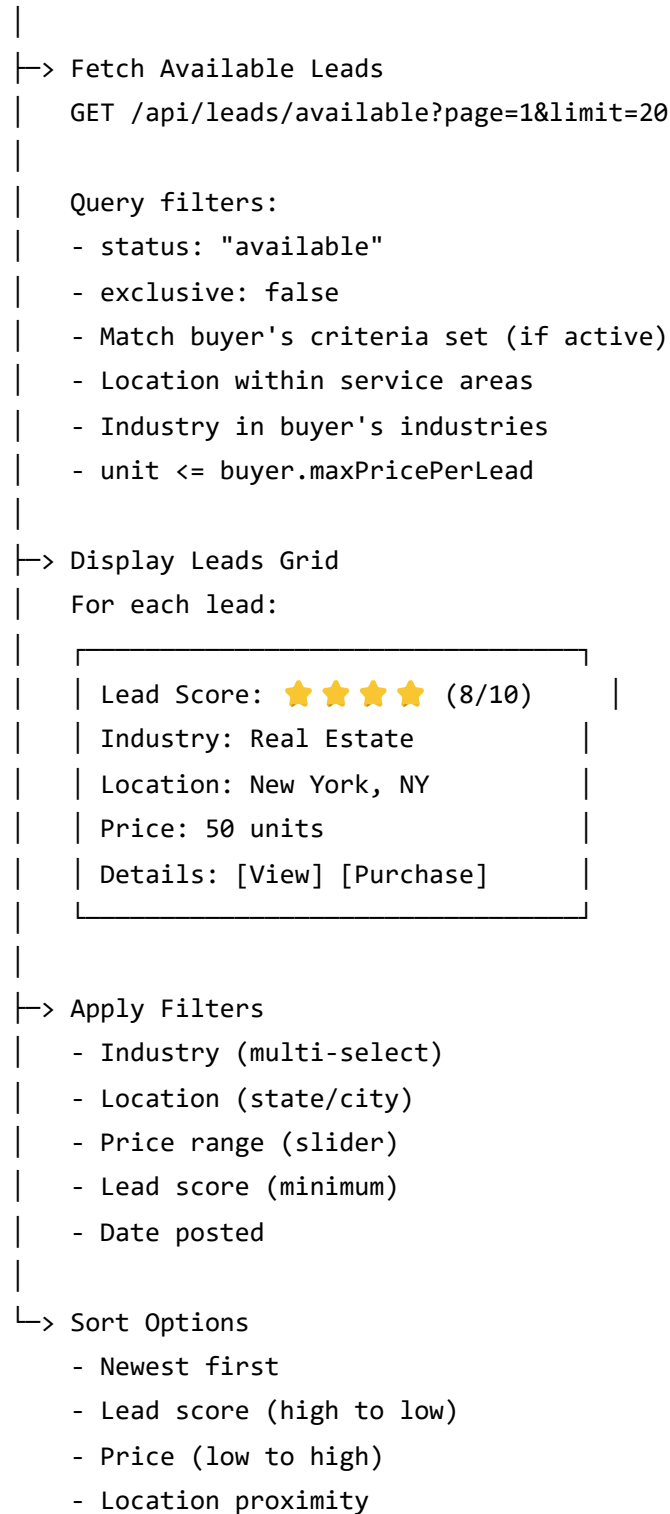
C. Lead Purchase Flow (Marketplace)

1. Buyer Browses Marketplace

Component: app/dashboard/buyer/marketplace/page.tsx

API: app/api/leads/available/route.ts

Buyer Dashboard → Marketplace



2. Purchase Lead

API: `app/api/leads/purchase/route.ts`

Buyer clicks "Purchase Lead"

```
|
|
|→ Lead Preview Modal
|   - Show partial information (name, company)
|   - Lead score and qualification details
|   - Price in units
|   - "Confirm Purchase" button
|
|→ Validate Purchase
|   |→ Check Buyer Wallet Balance
|   |   if (buyer.walletUnit < lead.unit) {
|   |       Show "Insufficient Units" error
|   |       Redirect to /purchaseUnit
|   |       return;
|   |   }
|   |
|   |→ Check Daily Limit
|   |   if (buyer.currentLeadsToday >= buyer.maxLeadsPerDay) {
|   |       Show "Daily limit reached" error
|   |       return;
|   |   }
|   |
|   |→ Check Lead Still Available
|   |   if (lead.status !== "available" || lead.soldCount >= lead.shareNumber) {
|   |       Show "Lead no longer available" error
|   |       return;
|   |   }
|   |
|→ Process Purchase Transaction
|   await dbConnect();
|   const session = await mongoose.startSession();
|   session.startTransaction();
|
|   try {
|       // 1. Deduct units from buyer
|       const updatedBuyer = await Buyer.findByIdAndUpdate(
|           buyerId,
|           {
|               $inc: {
|                   walletUnit: -lead.unit,
|                   currentLeadsToday: 1
|               },
|               $push: {
```

```

        purchaseHistory: {
            leadId: lead._id,
            date: new Date(),
            amount: lead.unit,
            unit: lead.unit
        }
    },
    { session, new: true }
);

// 2. Add units to seller
await User.findByIdAndUpdate(
    lead.userId,
    {
        $inc: { walletBalance: lead.unit }
    },
    { session }
);

// 3. Update lead
await Lead.findByIdAndUpdate(
    leadId,
    {
        status: "sold",
        $inc: { soldCount: 1 },
        $push: {
            soldTo: {
                buyerId,
                createdAt: new Date(),
                unit: lead.unit
            }
        }
    },
    { session }
);

// 4. Create transaction for buyer
await Transaction.create([
    {
        type: "lead_purchase",
        userId: buyerId,
        amount: lead.unit,
        currency: "USD",
    }
]);

```

```

    previousBalance: updatedBuyer.walletUnit + lead.unit,
    currentBalance: updatedBuyer.walletUnit,
    metadata: {
      leadId: lead._id,
      sellerId: lead.userId,
      unitsPurchased: lead.unit
    },
    paymentGateway: "internal",
    status: "completed",
    createdAt: new Date()
  }], { session });

```

// 5. Create transaction for seller

```

await Transaction.create([
  {
    type: "seller_income",
    userId: lead.userId,
    amount: lead.unit,
    currency: "USD",
    metadata: {
      leadId: lead._id,
      buyerId
    },
    paymentGateway: "internal",
    status: "completed",
    createdAt: new Date()
  }], { session });

```

// 6. Commit transaction

```

await session.commitTransaction();

```

```

} catch (error) {
  await session.abortTransaction();
  throw error;
} finally {
  session.endSession();
}

```

→ Send Notifications

- Notify buyer: "Lead purchased successfully"
- Notify seller: "Lead sold to {buyer.name}"
- Email receipts to both parties

→ Invalidate Caches

```
|   await invalidateAllUserSessions(buyerId);  
|   await invalidateAllUserSessions(lead.userId);  
|  
└─> Redirect to Lead Details  
    - Full lead information now visible  
    - Contact details accessible  
    - Option to add notes
```

3. Auto-Accept Flow

Automatic purchase based on criteria sets

New Qualified Lead Available in Marketplace

└> Find Buyers with Auto-Accept Enabled

```
const buyers = await Buyer.find({
  autoAcceptMatchingLeads: true,
  status: "active",
  activeCriteriaSetId: { $exists: true }
});
```

└> For Each Buyer, Check Criteria

```
for (const buyer of buyers) {
  const criteriaSet = buyer.criteriaSets.find(
    cs => cs._id.equals(buyer.activeCriteriaSetId)
  );

  const matches = checkCriteriaMatch(lead, criteriaSet);

  if (matches) {
    await autoPurchaseLead(lead._id, buyer._id);

    // Send notification
    await Notification.create({
      userId: buyer._id,
      type: "auto_purchase",
      message: `Auto-purchased lead: ${lead.name}`,
      leadId: lead._id
    });
  }
}
```

└> Criteria Matching Logic

```
function checkCriteriaMatch(lead, criteriaSet) {
  // Check lead types
  if (!criteriaSet.leadTypes.includes(lead.exclusive ? "exclusive" : "shared")) {
    return false;
  }

  // Check industries
  if (!criteriaSet.industries.includes(lead.industry)) {
    return false;
  }

  // Check location
```

```

const locationMatch = criteriaSet.locations.some(loc => {
  if (loc.city && loc.city === lead.location.city) return true;
  if (loc.state && loc.state === lead.location.state) return true;
  if (loc.zipCodes && loc.zipCodes.includes(lead.location.zipCode)) return true;
  if (loc.radius) {
    const distance = calculateDistance(
      buyer.location,
      lead.location
    );
    return distance <= loc.radius;
  }
  return false;
});

if (!locationMatch) return false;

// Check price
if (criteriaSet.maxPrice && lead.unit > criteriaSet.maxPrice) {
  return false;
}

// Check daily limit
if (criteriaSet.dailyLimit && buyer.currentLeadsToday >= criteriaSet.dailyLimit) {
  return false;
}

// Check excluded sources
if (criteriaSet.excludedSources.includes(lead.leadSource)) {
  return false;
}

return true;
}

```

D. Exclusive Lead Assignment Flow

Component: app/dashboard/seller/lead_management/page.tsx

API: app/api/leads/assignExclusiveLead/route.ts

Seller assigns exclusive lead to buyer(s)

```
|
|
|→ Mark Lead as Exclusive
|   lead.exclusive = true
|   lead.status = "assigned"
|
|
|→ Assign to Buyer(s)
|   lead.assignedTo = [{
|     buyerId,
|     accepted: false,
|     rejected: false,
|     assignedAt: new Date(),
|     expiresAt: new Date(Date.now() + 48*60*60*1000) // 48 hours
|   }]
|
|
|→ Buyer Receives Notification
|   "You have been assigned an exclusive lead"
|   - Email alert
|   - Dashboard notification
|   - SMS (if enabled)
|
|
|→ Buyer Views Lead
|   GET /api/buyers/myassignedleads
|   - See lead preview
|   - Lead score, industry, location
|   - Price in units
|   - Accept/Reject buttons
|
|
|→ Buyer Decision
|   |
|   |→ Accept Lead
|   |   POST /api/leads/accept
|   |   {
|   |     leadId,
|   |     buyerId,
|   |     action: "accept"
|   |   }
|   |
|   |
|   |   Process:
|   |   1. Check buyer.walletUnit >= lead.unit
|   |   2. If insufficient:
|   |       - Show "Add Funds" prompt
|   |       - Redirect to /purchaseUnit
```

- 3. If sufficient:
 - Deduct units from buyer
 - Add units to seller
 - Update lead.assignedTo[x].accepted = true
 - Update lead.status = "sold"
 - Create transactions
 - Grant full lead access
 - Send confirmation emails

↳ Reject Lead

POST /api/leads/reject

```
{  
  leadId,  
  buyerId,  
  action: "reject",  
  reason: "Not interested"  
}
```

Process:

1. Update lead.assignedTo[x].rejected = true
2. Notify seller of rejection
3. Seller can reassign to another buyer
4. Or make available in marketplace

↳ Expiration Handling (if no action within 48 hours)

Cron job checks:

- Find leads with expired assignments
- Mark as rejected
- Notify seller
- Suggest reassignment or marketplace listing

2. PAYMENT & SUBSCRIPTION FLOWS

A. Stripe Payment Integration

1. Buyer Purchases Units

Component: app/dashboard/buyer/purchaseUnit/page.tsx

API: app/api/payments/stripe/stripecheckoutapi/route.ts

Buyer Dashboard → Purchase Units

→ Display Unit Packages

Seller's configured packages:

100 Units - \$50
500 Units - \$200
1000 Units - \$350

→ Select Package

- Click on package
- Show total amount
- Payment method selection (Stripe/PayPal)

→ Create Stripe Checkout Session

POST /api/payments/stripe/stripecheckoutapi

```
{
  purchaseType: "credits",
  units: 100,
  amount: 50,
  sellerId: "...",
  buyerId: "..."
}
```

Server-side:

```
const session = await stripe.checkout.sessions.create({
  payment_method_types: ['card'],
  line_items: [{
    price_data: {
      currency: 'usd',
      product_data: {
        name: `${units} Units`,
        description: `Lead purchase credits`
      },
      unit_amount: amount * 100 // cents
    },
    quantity: 1
  }],
  mode: 'payment',
  success_url: `${FRONTEND_URL}/dashboard/buyer/success?session_id={CHECKOUT_SESSION_ID}`,
  cancel_url: `${FRONTEND_URL}/dashboard/buyer/purchaseUnit`,
  metadata: {
```

```

    purchaseType: "credits",
    units,
    sellerId,
    buyerId,
    userId: buyerId
  },
  // Stripe Connect: Transfer to seller's account
  payment_intent_data: {
    application_fee_amount: amount * 0.05 * 100, // 5% platform fee
    transfer_data: {
      destination: seller.stripeAccountId
    }
  }
});

```

↳ Redirect to Stripe Checkout

```
window.location.href = session.url;
```

↳ Buyer Completes Payment

- Enter card details
- 3D Secure verification (if required)
- Confirm payment

↳ Stripe Webhook Triggered

```
POST /api/payments/stripe/stripewebhook
```

```
Event: checkout.session.completed
```

```
const session = event.data.object;
```

```
const metadata = session.metadata;
```

```

if (metadata.purchaseType === "credits") {
  // Add units to buyer
  await Buyer.findByIdAndUpdate(metadata.buyerId, {
    $inc: { walletUnit: parseInt(metadata.units) }
  });
}

```

```
// Create transaction record
```

```

await Transaction.create({
  type: "units_purchase",
  userId: metadata.buyerId,
  amount: session.amount_total / 100,
  currency: session.currency,
  metadata: {

```

```

        unitsPurchased: metadata.units,
        sellerId: metadata.sellerId,
        buyerId: metadata.buyerId
    },
    paymentGateway: "stripe",
    gatewayTransactionId: session.payment_intent,
    status: "completed",
    createdAt: new Date()
  });

  // Send confirmation email
  await sendEmail({
    to: buyer.email,
    subject: "Units Purchase Successful",
    template: "unit-purchase-confirmation",
    data: { units: metadata.units, amount: session.amount_total / 100 }
  });

  // Invalidate cache
  await invalidateAllUserSessions(metadata.buyerId);
}

```

- ↳ Redirect to Success Page
- Show confirmation message
 - Display new wallet balance
 - Suggest browsing marketplace

2. Subscription Payment

Component: app/plan/page.tsx

API: app/api/payments/stripe/stripecheckoutapi/route.ts

Seller selects subscription tier

└─> Display Available Tiers

GET /api/tiers

Basic Plan - \$49/month
- 100 leads/month
- 5 forms
- 10 buyers
[Subscribe]
Pro Plan - \$99/month
- 500 leads/month
- 20 forms
- 50 buyers
- Call recording
[Subscribe]

└─> Select Duration

- Monthly (\$99)

- Quarterly (\$270 - 9% off)

- Annual (\$990 - 17% off)

└─> Create Checkout Session

POST /api/payments/stripe/stripecheckoutapi

```
{
  purchaseType: "subscription",
  tierId: "...",
  tierName: "Pro Plan",
  durationMonths: 1,
  amount: 99
}
```

Server-side:

```
const tier = await Tier.findById(tierId);
```

```
const session = await stripe.checkout.sessions.create({
```

```
  payment_method_types: ['card'],
```

```
  line_items: [{
```

```
    price: tier.stripePriceId, // Pre-configured in Stripe
```

```
    quantity: durationMonths
```

```

    }],
    mode: 'payment',
    success_url: `${FRONTEND_URL}/dashboard/seller/success`,
    cancel_url: `${FRONTEND_URL}/plan`,
    metadata: {
      purchaseType: "subscription",
      tierId,
      tierName: tier.name,
      tierType: tier.tierType,
      durationMonths,
      userId: session.user.id
    }
  });

```

→ Payment Completed

→ Webhook Handler

```
Event: checkout.session.completed
```

```
const metadata = session.metadata;
```

```

if (metadata.purchaseType === "subscription") {
  const expiryDate = new Date();
  expiryDate.setMonth(expiryDate.getMonth() + parseInt(metadata.durationMonths));

```

```
// Update user subscription
```

```

await User.findByIdAndUpdate(metadata.userId, {
  'subscription.subscriptionTierId': metadata.tierId,
  'subscription.isSubscriptionActive': true,
  'subscription.subscriptionStartDate': new Date(),
  'subscription.subscriptionExpiryDate': expiryDate,
  'subscription.subscriptionPaymentMethod': 'stripe',
  'subscription.autoRenew': true
});

```

```
// Create transaction
```

```

await Transaction.create({
  type: "subscription_payment",
  userId: metadata.userId,
  amount: session.amount_total / 100,
  currency: session.currency,
  metadata: {
    tierId: metadata.tierId,

```

```

    tierName: metadata.tierName,
    tierType: metadata.tierType,
    subscriptionPlan: metadata.tierName,
    subscriptionDuration: `${metadata.durationMonths} month(s)`,
    userEmail: user.email
  },
  paymentGateway: "stripe",
  gatewayTransactionId: session.payment_intent,
  status: "completed"
});

// Invalidate cache to refresh session
await invalidateAllUserSessions(metadata.userId);

// Send welcome email
await sendEmail({
  to: user.email,
  subject: `Welcome to ${metadata.tierName}!`,
  template: "subscription-activated",
  data: { tierName: metadata.tierName, features: tier.features }
});
}

```

↳ Grant Tier Features

- User can now access tier-limited features
- Dashboard shows active subscription badge

3. Seller Payout (Stripe Connect)

Component: app/dashboard/seller/payouts/page.tsx

API: app/api/payments/payout/stripe/route.ts

Seller requests payout

```
|  
|  
|→ Check Stripe Account Status  
|   if (!seller.stripeAccountId || !seller.stripeOnboarded) {  
|     Show "Complete Stripe onboarding" prompt  
|     Redirect to /api/payments/stripe/onboard  
|     return;  
|   }  
|
```

```
|  
|→ Display Payout Form  
|   - Current wallet balance: $500  
|   - Minimum payout: $10  
|   - Enter payout amount  
|   - [Request Payout] button  
|
```

```
|  
|→ Submit Payout Request  
|   POST /api/payments/payout/stripe  
|   {  
|     amount: 500,  
|     userId: sellerId  
|   }  
|
```

```
|   Server-side validation:  
|   - Check seller.walletBalance >= amount  
|   - Check amount >= minimumPayout (10)  
|   - Check seller.stripeAccountId exists  
|
```

```
|  
|→ Create Stripe Transfer  
|   const transfer = await stripe.transfers.create({  
|     amount: amount * 100, // cents  
|     currency: 'usd',  
|     destination: seller.stripeAccountId,  
|     description: `Payout to ${seller.name}`,  
|     metadata: {  
|       userId: sellerId,  
|       type: "payout"  
|     }  
|   });  
|
```

```
|  
|→ Update Seller Balance  
|   await User.findByIdAndUpdate(sellerId, {  
|     $inc: { walletBalance: -amount }  
|   });  
|
```

└> Create Transaction Record

```
await Transaction.create({
  type: "seller_payout",
  userId: sellerId,
  amount,
  currency: "USD",
  previousBalance: seller.walletBalance,
  currentBalance: seller.walletBalance - amount,
  metadata: {
    sellerId,
    stripeTransferId: transfer.id,
    transferVerified: true,
    transferAmount: amount
  },
  paymentGateway: "stripe",
  gatewayTransactionId: transfer.id,
  status: "completed"
});
```

└> Stripe Processes Transfer

- Funds typically arrive in 2-7 business days
- Depends on bank and country

└> Webhook: transfer.paid

POST /api/payments/stripe/stripewebhook

Event: transfer.paid

// Mark transfer as verified

```
await Transaction.findOneAndUpdate(
  { gatewayTransactionId: event.data.object.id },
  { 'metadata.transferVerified': true }
);
```

└> Send Confirmation

- Email: "Payout initiated - \$500"
- Expected arrival date
- Transaction ID for reference

B. PayPal Payment Integration

1. Buyer Purchases Units (PayPal)

API: `app/api/payments/paypal/create/route.ts` , `app/api/payments/paypal/capture/route.ts`

Buyer selects PayPal payment method

└> Create PayPal Order

POST /api/payments/paypal/create

```
{
  units: 100,
  amount: 50,
  sellerId,
  buyerId
}
```

Server-side:

```
const paypalOrder = await paypal.orders.create({
  intent: 'CAPTURE',
  purchase_units: [{
    amount: {
      currency_code: 'USD',
      value: amount.toString()
    },
    description: `${units} Lead Units`,
    custom_id: JSON.stringify({
      purchaseType: "credits",
      units,
      sellerId,
      buyerId
    })
  }],
  application_context: {
    return_url: `${FRONTEND_URL}/dashboard/buyer/paypal-success`,
    cancel_url: `${FRONTEND_URL}/dashboard/buyer/purchaseUnit`
  }
});
```

└> Redirect to PayPal

```
const approvalUrl = paypalOrder.links.find(l => l.rel === 'approve').href;
window.location.href = approvalUrl;
```

└> Buyer Approves Payment on PayPal

└> Return to App

Redirect to return_url with ?token={orderId}

└> Capture Payment

```

POST /api/payments/paypal/capture
{ orderId }

const capture = await paypal.orders.capture(orderId);
const customId = JSON.parse(capture.purchase_units[0].custom_id);

// Add units to buyer
await Buyer.findByIdAndUpdate(customId.buyerId, {
  $inc: { walletUnit: customId.units }
});

// Create transaction
await Transaction.create({
  type: "units_purchase",
  userId: customId.buyerId,
  amount: parseFloat(capture.purchase_units[0].amount.value),
  currency: capture.purchase_units[0].amount.currency_code,
  metadata: {
    unitsPurchased: customId.units,
    sellerId: customId.sellerId,
    buyerId: customId.buyerId
  },
  paymentGateway: "paypal",
  gatewayTransactionId: capture.id,
  status: "completed"
});

```

↳ Show Success Message

2. Seller Payout (PayPal)

API: app/api/payments/payout/paypal/route.ts

Seller requests PayPal payout

```
|  
|  
|→ Check PayPal Account Linked  
|   if (!seller.paypalCustomerId) {  
|       Show "Link PayPal account" prompt  
|       return;  
|   }  
|  
|→ Create Payout  
|   POST /api/payments/payout/paypal  
|   { amount: 500 }  
|  
|   const payout = await paypal.payouts.create({  
|       sender_batch_header: {  
|         sender_batch_id: `payout_${Date.now()}`,  
|         email_subject: 'You have a payout from BRIXCOT',  
|         email_message: 'You have received a payout from lead sales'  
|       },  
|       items: [{  
|         recipient_type: 'EMAIL',  
|         amount: {  
|           value: amount.toString(),  
|           currency: 'USD'  
|         },  
|         receiver: seller.paypalCustomerId,  
|         note: 'Lead sales payout',  
|         sender_item_id: `payout_item_${Date.now()}`  
|       }]  
|     });  
|  
|   // Deduct from seller balance  
|   await User.findByIdAndUpdate(sellerId, {  
|     $inc: { walletBalance: -amount }  
|   });  
|  
|   // Create transaction  
|   await Transaction.create({  
|     type: "seller_payout",  
|     userId: sellerId,  
|     amount,  
|     currency: "USD",  
|     metadata: {  
|       sellerId,
```

```
|      payoutId: payout.batch_header.payout_batch_id  
|    },  
|    paymentGateway: "paypal",  
|    gatewayTransactionId: payout.batch_header.payout_batch_id,  
|    status: "completed"  
|  });  
|
```

↳ PayPal Sends Funds

- Funds arrive in seller's PayPal account within 24 hours

C. Subscription Management

1. Tier Limits Enforcement

Middleware: Used across all resource creation endpoints

When seller creates resource (form, lead, buyer, etc.):

```
|
|
|→ Fetch User with Subscription
|   const user = await User.findById(userId).populate('subscription.subscriptionTierId');
|   const tier = user.subscription.subscriptionTierId;
|
|
|→ Check if Subscription Active
|   if (!user.subscription.isSubscriptionActive) {
|       tier = await Tier.findOne({ tierType: 'free' });
|   }
|
|   if (new Date() > user.subscription.subscriptionExpiryDate) {
|       // Subscription expired
|       await User.findByIdAndUpdate(userId, {
|           'subscription.isSubscriptionActive': false
|       });
|       tier = await Tier.findOne({ tierType: 'free' });
|   }
|
|
|→ Get Current Usage
|   GET /api/subscriptions/limits
|
|   const usage = {
|       forms: user.forms.length,
|       buyers: user.buyers.length,
|       leads: user.leads.length,
|       twilioNumbers: user.trackingNumbers.length
|   };
|
|
|→ Check Against Tier Limits
|   const limits = tier.tierLimits;
|
|   switch (resourceType) {
|       case 'form':
|           if (usage.forms >= limits.forms) {
|               return {
|                   allowed: false,
|                   error: `Form limit reached (${limits.forms}/${limits.forms})`,
|                   upgradeRequired: true
|               };
|           }
|           break;
```

```

|     case 'buyer':
|         if (usage.buyers >= limits.buyers) {
|             return {
|                 allowed: false,
|                 error: `Buyer limit reached (${limits.buyers}/${limits.buyers})`,
|                 upgradeRequired: true
|             };
|         }
|         break;
|
|         // ... similar for other resources
|     }

```

└─> If Limit Exceeded

- Return error response
- Frontend shows upgrade modal
- Suggest higher tier
- "Upgrade Now" button → /plan

└─> If Within Limits

- Allow resource creation
- Return success

2. Subscription Renewal (Cron Job)

Scheduled Task: Runs daily at midnight

Cron job: 0 0 * * * (daily at midnight)

```
|
|
|→ Find Subscriptions Expiring Soon
|  const upcomingExpirations = await User.find({
|    'subscription.isSubscriptionActive': true,
|    'subscription.subscriptionExpiryDate': {
|      $lte: new Date(Date.now() + 7*24*60*60*1000), // Within 7 days
|      $gte: new Date() // Not yet expired
|    }
|  });
|
|→ For Each User
|  for (const user of upcomingExpirations) {
|    const daysUntilExpiry = Math.ceil(
|      (user.subscription.subscriptionExpiryDate - new Date()) / (1000*60*60*24)
|    );
|
|    // Send reminder emails
|    if (daysUntilExpiry === 7 && user.subscription.notificationsSent === 0) {
|      await sendEmail({
|        to: user.email,
|        subject: "Your subscription expires in 7 days",
|        template: "subscription-renewal-reminder",
|        data: {
|          expiryDate: user.subscription.subscriptionExpiryDate,
|          renewalLink: `${FRONTEND_URL}/plan`
|        }
|      });
|
|      await User.findByIdAndUpdate(user._id, {
|        $inc: { 'subscription.notificationsSent': 1 },
|        'subscription.lastNotificationDate': new Date()
|      });
|    }
|
|    if (daysUntilExpiry === 3 && user.subscription.notificationsSent === 1) {
|      await sendEmail({
|        to: user.email,
|        subject: "Your subscription expires in 3 days",
|        template: "subscription-renewal-urgent"
|      });
|
|      await User.findByIdAndUpdate(user._id, {
```



```

    $inc: { 'subscription.notificationsSent': 1 }
  });
}
}

```

→ Find Expired Subscriptions

```

const expired = await User.find({
  'subscription.isSubscriptionActive': true,
  'subscription.subscriptionExpiryDate': { $lt: new Date() }
});

```

→ Handle Expiration

```

for (const user of expired) {
  if (user.subscription.autoRenew) {
    // Attempt auto-renewal
    try {
      const tier = await Tier.findById(user.subscription.subscriptionTierId);

      if (user.subscription.subscriptionPaymentMethod === 'stripe') {
        // Charge saved payment method
        const charge = await stripe.charges.create({
          amount: tier.renewalPrice * 100,
          currency: 'usd',
          customer: user.stripeCustomerId,
          description: `Subscription renewal - ${tier.name}`
        });

        if (charge.status === 'succeeded') {
          // Extend subscription
          const newExpiryDate = new Date();
          newExpiryDate.setMonth(newExpiryDate.getMonth() + 1);

          await User.findByIdAndUpdate(user._id, {
            'subscription.subscriptionExpiryDate': newExpiryDate,
            'subscription.notificationsSent': 0
          });

          // Create transaction
          await Transaction.create({
            type: "subscription_renewal",
            userId: user._id,
            amount: tier.renewalPrice,
            currency: "USD",

```

```

        metadata: {
            tierId: tier._id,
            tierName: tier.name
        },
        paymentGateway: "stripe",
        gatewayTransactionId: charge.id,
        status: "completed"
    });

    // Send success email
    await sendEmail({
        to: user.email,
        subject: "Subscription renewed successfully",
        template: "subscription-renewed"
    });
}
}
} catch (error) {
    console.error('Auto-renewal failed:', error);

    // Deactivate subscription
    await User.findByIdAndUpdate(user._id, {
        'subscription.isSubscriptionActive': false
    });

    // Send payment failure email
    await sendEmail({
        to: user.email,
        subject: "Subscription renewal failed",
        template: "subscription-payment-failed",
        data: { error: error.message }
    });
}
} else {
    // Auto-renew disabled - deactivate
    await User.findByIdAndUpdate(user._id, {
        'subscription.isSubscriptionActive': false
    });

    // Send expiration notice
    await sendEmail({
        to: user.email,
        subject: "Your subscription has expired",

```

```
|
└─> Log Results
```

Setup Flow

API: `app/api/integrations/zapier/route.ts`

Seller navigates to Integrations → Zapier

└─> Display Zapier Connection Form

- Zapier Webhook URL input
- Event selection checkboxes:
 - ☐ Lead Created
 - ☐ Lead Updated
 - ☐ Lead Qualified
 - ☐ Lead Accepted
 - ☐ Lead Rejected
 - ☐ Buyer Assigned
 - ☐ Call Completed

└─> Submit Configuration

POST /api/integrations/zapier

```
{
  name: "My Zapier Integration",
  url: "https://hooks.zapier.com/hooks/catch/12345/abcdef/",
  events: {
    leadCreated: true,
    leadUpdated: false,
    leadQualified: true,
    leadAccepted: true,
    leadRejected: false,
    buyerAssigned: true,
    callCompleted: false
  },
  headers: {
    "Content-Type": "application/json"
  }
}
```

Server-side:

// Generate webhook secret

```
const secret = crypto.randomBytes(32).toString('hex');
```

// Create webhook config

```
const webhookConfig = await WebhookConfig.create({
  userId: session.user.id,
  name,
  url,
  source: "zapier",
  secret,
```

```

    isActive: true,
    events,
    headers,
    maxRetriesPerEvent: 3,
    retryDelaySeconds: 60,
    ratelimit: {
      maxPerMinute: 60,
      maxPerHour: 1000
    }
  });

```

└─> Test Connection

```

// Send test webhook
const testPayload = {
  type: "test",
  action: "connection_test",
  data: {
    message: "Zapier integration connected successfully",
    timestamp: new Date().toISOString()
  },
  source: "brixcot"
};

const signature = crypto
  .createHmac('sha256', secret)
  .update(JSON.stringify(testPayload))
  .digest('hex');

const response = await fetch(url, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-Webhook-Signature': signature,
    'X-Webhook-Timestamp': Date.now().toString(),
    ...headers
  },
  body: JSON.stringify(testPayload)
});

if (response.ok) {
  return { success: true, message: "Connection successful" };
} else {
  throw new Error("Connection test failed");
}

```

```
|  }  
|  
└─> Save Configuration  
    - Show success message  
    - Display webhook secret (one-time)  
    - List active integrations
```

Trigger Flow

Helper: lib/integrations/zapierTrigger.ts

Event occurs in system (e.g., new lead created)

└─> Find Active Zapier Configs

```
const configs = await WebhookConfig.find({
  userId: sellerId,
  source: "zapier",
  isActive: true,
  'events.leadCreated': true
});
```

└─> For Each Config, Send Webhook

```
for (const config of configs) {
  await sendWebhook(config, lead);
}
```

```
async function sendWebhook(config, lead) {
  // Prepare payload
  const payload = {
    type: "lead",
    action: "created",
    data: {
      leadId: lead._id,
      name: lead.name,
      email: lead.email,
      phone: lead.phone,
      company: lead.company,
      industry: lead.industry,
      leadScore: lead.leadScore,
      qualificationScore: lead.qualificationScore,
      status: lead.status,
      location: lead.location,
      createdAt: lead.createdAt
    },
    source: "brixcot",
    timestamp: new Date().toISOString()
  };

  // Generate signature
  const signature = crypto
    .createHmac('sha256', config.secret)
    .update(JSON.stringify(payload))
    .digest('hex');
```

```

// Send webhook with retry logic
let attempt = 0;
let success = false;
let lastError = null;

while (attempt <= config.maxRetriesPerEvent && !success) {
  try {
    const startTime = Date.now();

    const response = await fetch(config.url, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'X-Webhook-Signature': signature,
        'X-Webhook-Timestamp': Date.now().toString(),
        ...config.headers
      },
      body: JSON.stringify(payload),
      timeout: 30000 // 30 seconds
    });

    const responseTime = Date.now() - startTime;

    if (response.ok) {
      success = true;

      // Update stats
      await WebhookConfig.findByIdAndUpdate(config._id, {
        $inc: {
          totalDispatched: 1,
          successCount: 1
        },
        lastDispatchedAt: new Date(),
        $push: {
          dispatchLogs: {
            $each: [{
              timestamp: new Date(),
              success: true,
              statusCode: response.status,
              retryCount: attempt,
              responseTime
            }],
            $slice: -100 // Keep last 100 logs
          }
        }
      });
    }
  } catch (error) {
    lastError = error;
    attempt++;
  }
}

```



```

        }
    }
    });
    } else {
        throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }
} catch (error) {
    lastError = error.message;
    attempt++;

    if (attempt <= config.maxRetriesPerEvent) {
        // Wait before retry (exponential backoff)
        const delay = config.retryDelaySeconds * Math.pow(2, attempt - 1) * 1000;
        await new Promise(resolve => setTimeout(resolve, delay));
    }
}
}

if (!success) {
    // All retries failed
    await WebhookConfig.findByIdAndUpdate(config._id, {
        $inc: {
            totalDispatched: 1,
            failureCount: 1
        },
        lastDispatchedAt: new Date(),
        lastErrorAt: new Date(),
        lastErrorMessage: lastError,
        $push: {
            dispatchLogs: {
                $each: [{
                    timestamp: new Date(),
                    success: false,
                    errorMessage: lastError,
                    retryCount: attempt,
                    responseTime: 0
                }],
                $slice: -100
            }
        }
    });

    // Notify seller of failure

```

```
|      await Notification.create({  
|        userId: config.userId,  
|        type: "webhook_failure",  
|        message: `Zapier webhook failed after ${attempt} attempts: ${lastError}`  
|      });  
|    }  
|  }  
|}
```

└─> Rate Limiting

- Track webhooks sent per minute/hour
- If limit exceeded, queue for later
- Use Redis for distributed rate limiting

B. HubSpot Integration

Component: app/dashboard/seller/integrations/hubspot/page.tsx

Adapter: lib/integrations/hubspot/hubspotAdapter.ts

Connection Setup

Seller navigates to Integrations → HubSpot

```
|
|
|→ OAuth Flow
|  1. Click "Connect HubSpot"
|  2. Redirect to HubSpot OAuth:
|     https://app.hubspot.com/oauth/authorize
|     ?client_id={CLIENT_ID}
|     &redirect_uri={REDIRECT_URI}
|     &scope=crm.objects.contacts.write crm.objects.deals.write
|
|  3. User authorizes app
|
|  4. HubSpot redirects back with code:
|     {REDIRECT_URI}?code={AUTH_CODE}
|
|  5. Exchange code for access token:
|     POST https://api.hubapi.com/oauth/v1/token
|     {
|       grant_type: "authorization_code",
|       client_id,
|       client_secret,
|       redirect_uri,
|       code: AUTH_CODE
|     }
|
|  6. Store tokens (encrypted):
|     await User.findByIdAndUpdate(userId, {
|       'integrations.hubspot': {
|         accessToken: encrypt(tokens.access_token),
|         refreshToken: encrypt(tokens.refresh_token),
|         expiresAt: new Date(Date.now() + tokens.expires_in * 1000),
|         isActive: true
|       }
|     });
|
|→ Test Connection
|   - Fetch HubSpot account info
|   - Display connected portal name
|   - Show "Connected ✓" badge
```

Lead Sync to HubSpot

Trigger: New lead created or updated

Lead created/updated in BRIXCOT

└─> Check HubSpot Integration Active

```
if (!user.integrations.hubspot.isActive) return;
```

└─> Prepare Contact Data

```
const contactData = {
  properties: {
    email: lead.email,
    firstname: lead.name.split(' ')[0],
    lastname: lead.name.split(' ').slice(1).join(' ') || '',
    phone: lead.phone,
    company: lead.company,

    // Custom properties
    lead_score: lead.leadScore,
    lead_source: lead.leadSource,
    industry: lead.industry,
    city: lead.location.city,
    state: lead.location.state,
    zip: lead.location.zipCode,

    // BRIXCOT-specific
    brixcot_lead_id: lead._id.toString(),
    brixcot_qualification_score: lead.qualificationScore
  }
};
```

└─> Create or Update Contact

```
try {
  // Check if contact exists
  const existingContact = await hubspotClient.crm.contacts.searchApi.doSearch({
    filterGroups: [{
      filters: [{
        propertyName: 'email',
        operator: 'EQ',
        value: lead.email
      }]
    }]
  });

  let contactId;
```

```

|   if (existingContact.total > 0) {
|       // Update existing contact
|       contactId = existingContact.results[0].id;
|
|       await hubspotClient.crm.contacts.basicApi.update(
|           contactId,
|           contactData
|       );
|   } else {
|       // Create new contact
|       const createdContact = await hubspotClient.crm.contacts.basicApi.create(
|           contactData
|       );
|       contactId = createdContact.id;
|   }
|
|   // Store mapping
|   await Lead.findByIdAndUpdate(lead._id, {
|       'metadata.hubspotContactId': contactId
|   });
|
| } catch (error) {
|   console.error('HubSpot sync error:', error);
|
|   // Log error in webhook config
|   await WebhookConfig.findOneAndUpdate(
|       { userId: user._id, source: 'hubspot' },
|       {
|         $inc: { failureCount: 1 },
|         lastErrorAt: new Date(),
|         lastErrorMessage: error.message
|       }
|   );
| }

```

↳ Create Deal (if qualified)

```

|   if (lead.status === 'qualified' && !lead.metadata.hubspotDealId) {
|       const dealData = {
|         properties: {
|           dealname: `${lead.company || lead.name} - ${lead.industry}`,
|           amount: lead.unit || 0,
|           dealstage: 'leadqualified',
|           pipeline: 'default',

```

```

    closedate: new Date(Date.now() + 30*24*60*60*1000).toISOString(), // 30 days
    hubspot_owner_id: user.integrations.hubspot.ownerId,

    // Custom properties
    lead_source: lead.leadSource,
    brixcot_lead_id: lead._id.toString()
  },
  associations: [{
    to: { id: contactId },
    types: [{ associationCategory: 'HUBSPOT_DEFINED', associationTypeId: 3 }] // Contact 1
  }]
};

const deal = await hubspotClient.crm.deals.basicApi.create(dealData);

await Lead.findByIdAndUpdate(lead._id, {
  'metadata.hubspotDealId': deal.id
});
}

```

Bi-directional Sync (Webhooks)

API: `app/api/integrations/hubspot/webhook/route.ts`

HubSpot sends webhook to BRIXCOT

```
|
|
|→ Verify Webhook Signature
|   const signature = req.headers['x-hubspot-signature'];
|   const requestBody = await req.text();
|
|   const expectedSignature = crypto
|     .createHmac('sha256', HUBSPOT_CLIENT_SECRET)
|     .update(requestBody)
|     .digest('hex');
|
|   if (signature !== expectedSignature) {
|     return NextResponse.json({ error: 'Invalid signature' }, { status: 401 });
|   }
|
|→ Parse Event
|   const events = JSON.parse(requestBody);
|
|   for (const event of events) {
|     switch (event.subscriptionType) {
|       case 'contact.propertyChange':
|         await handleContactUpdate(event);
|         break;
|
|       case 'deal.propertyChange':
|         await handleDealUpdate(event);
|         break;
|
|       case 'contact.deletion':
|         await handleContactDeletion(event);
|         break;
|     }
|   }
|
|→ Handle Contact Update
|   async function handleContactUpdate(event) {
|     const contactId = event.objectId;
|     const propertyName = event.propertyName;
|     const propertyValue = event.propertyValue;
|
|     // Find lead by HubSpot contact ID
|     const lead = await Lead.findOne({
|       'metadata.hubspotContactId': contactId
```



```

});

if (!lead) return;

// Sync specific property changes
switch (propertyName) {
  case 'lifecyclestage':
    if (propertyValue === 'customer') {
      await Lead.findByIdAndUpdate(lead._id, {
        status: 'converted'
      });
    }
    break;

  case 'phone':
    await Lead.findByIdAndUpdate(lead._id, {
      phone: propertyValue
    });
    break;

  case 'company':
    await Lead.findByIdAndUpdate(lead._id, {
      company: propertyValue
    });
    break;
}
}

```

C. Salesforce Integration

Similar pattern to HubSpot with Salesforce-specific API calls.

Key Differences:

- Uses Salesforce REST API v59.0
- Lead object instead of Contact
- Convert Lead to Contact/Opportunity flow
- Custom object mapping for BRXCOT fields

End of Document 2

Next Document: Component Structure, Technical Features, and Deployment Architecture